

BACHELORS IN COMPUTER SYSTEMS ENGINEERING**Course Code: CS-324****Course Title: Machine Learning****Complex Engineering Problem****TE Batch 2023, Spring Semester 2026****Grading Rubric 1: Final Model Implementation and Evaluation (Milestone 3)****TERM PROJECT****Group Members:**

Student No.	Name	Roll No.
S1	Shafia Arif	CS-23107
S2	Sania Shakeel	CS-23108
S3	Muneeba	CS-23109

CRITERIA AND SCALES				Marks Obtained			
				S1	S2	S3	S4
Criterion 1: Model implementation and fulfillment of technical requirements. (CPA-1, CPA-3) [3 marks]							
1	2	3	4				
Required models are not implemented or are incorrectly implemented.	Some required models are implemented, but with major issues.	All required models are implemented correctly with minor issues.	All required models are correctly implemented and fully meet the project requirements.				
Criterion 2: Comparative analysis, evaluation, and discussion. (CPA-1, CPA-2) [2 marks]							
1	2	3	4				
No meaningful evaluation or comparison is provided.	Limited evaluation and comparison with weak analysis.	Adequate evaluation and comparison with reasonable analysis.	Thorough evaluation, clear comparisons, and insightful discussion of results.				
Criterion 3: Report quality and adherence to format. [2 marks]							
1	2	3	4				
The report is poorly written and does not follow the given format.	The report partially follows the format and lacks clarity.	The report follows the format with minor formatting or clarity issues.	The report is clear, well written, and fully adheres to the given format and requirements.				
Criterion 4: Individual contribution, teamwork, and viva performance. (CPA-1, CPA-2, CPA-3) [3 marks]							
1	2	3	4				
The student did not contribute meaningfully and could not explain the work.	The student contributed partially and showed limited understanding.	The student contributed satisfactorily and explained the work adequately.	The student demonstrated strong contribution, teamwork, and excellent understanding during viva.				

Final Score = (Criteria_1_score) + (Criteria_2_score) + (Criteria_3_score) = _____

Introduction

This project presents a complete end-to-end supervised machine learning system designed for binary image classification of consumer product packaging. The system classifies packaging images into two categories: Good Packaging and Defected Packaging. The dataset was self-collected through visits to real-world retail environments, including Imtiaz Supermarket, local marts, and neighborhood general stores in Karachi, ensuring the data reflects genuine real-world conditions.

Dataset Description

The dataset consists of RGB images of consumer product packaging collected from real-world environments. Images include a wide variety of packaging types such as bottles, cartons, sachets, biscuit boxes, pasta packets, juice cartons, and household product containers. The dataset captures real-world variations including:

- Different lighting conditions (natural daylight, artificial store lighting, mixed).
- Diverse backgrounds (store shelves, home counters, tables).
- Multiple viewing angles (front, side, top, oblique).
- Varying image resolutions due to different mobile phone cameras.

Each image belongs to one of the following two classes:

- Good Packaging (Label = 0): Packaging is fully intact with no visible damage, tears, dents, or deformations.
- Defected Packaging (Label = 1): Packaging shows any visible damage such as tears, dents, crushed areas, broken seals, leakage, or wrinkles.

Data Collection Method and Sources

The dataset was collected through on-site visits to multiple real-world retail environments in Karachi. Data collection locations included:

- **Imtiaz Supermarket** — for a wide variety of branded packaged goods.
- **Al-Jadeed Store** — for local and imported product packaging.
- **Neighborhood general stores** — for everyday packaged items.
- **Home environment** — for packaging diversity and increased variation in backgrounds.

Images were captured using mobile phone cameras under varying conditions to simulate realistic deployment scenarios. Packaging defects were either naturally occurring (found on store shelves or at home) or lightly induced to represent real-world damage conditions such as dents, tears, wrinkles, and broken seals. All images were manually reviewed and labeled based on visual inspection.

Dataset Statistics

Statistic	Good Packaging (Class 0)	Defected Packaging (Class 1)	Total / Overall
Number of Images	~329	~328	657
Image Dimensions (after resize)	128 × 128	128 × 128	128 × 128
Color Channels	1 (Grayscale)	1 (Grayscale)	1 (Grayscale)
Augmentation Applied	Flip, Brightness, Contrast	Flip, Brightness, Contrast	Yes
Task Type	Binary Classification	Binary Classification	Binary

Labeling Methodology

All images were manually labeled based on visual inspection of packaging condition:

- **Good Packaging (Label = 0):** No visible defects; packaging is fully intact and undamaged. Printed labels are clear, seals are closed, and no physical deformations are present.
- **Defected Packaging (Label = 1):** Any visible damage is present, including small dents, tears, wrinkles, broken seals, crushed areas, leakage stains, or large deformations.

This binary labeling ensures a clear decision boundary between acceptable and unacceptable packaging quality, suitable for a binary classification task in a quality control pipeline.

Preprocessing Pipeline

To prepare the dataset for machine learning, a systematic preprocessing pipeline was implemented using Python libraries including TensorFlow, NumPy, Pillow (PIL), and OpenCV.

2.1 Image Loading and Inventory

The dataset was organized into two folders corresponding to each class: `good_packaging` and `defected_packaging`. A loop was used to iterate through each folder, read all image file paths, and store them in a Pandas DataFrame along with their class labels. This created a structured inventory of the entire dataset with columns for file path, class label (string), and integer label encoding.

2.2 Data Cleaning

A try-except block was used during image loading to handle corrupted or unreadable image files. Any image causing an error was skipped, ensuring that only valid, readable images were retained in the dataset. This step also filtered out any non-image files inadvertently placed in the data folders.

2.3 Color Space and Resizing

Images were loaded using TensorFlow's `tf.io.read_file` and `tf.image.decode_jpeg` functions, which automatically read images in RGB format. All images were resized to a fixed dimension of 128 × 128 pixels using `tf.image.resize`. This ensures uniform input dimensions required for neural network models. While 224 × 224 is the standard for VGG16 and other networks, 128 × 128 was chosen.

2.4 Label Encoding

Class labels were encoded as integers using a dictionary mapping:

- good_packaging ==> 0
- defected_packaging ==> 1

Binary integer labels were stored in a dedicated column (label_int) in the DataFrame and used as targets throughout model training and evaluation.

2.5 Pixel Normalization

To improve training stability and convergence speed, pixel values were normalized from the range [0, 255] to [0, 1] by dividing by 255.0 in the TF data pipeline:

```
img = tf.cast(img, tf.float32) / 255.0
```

This normalization prevents large gradient updates and ensures consistent feature scales across all input dimensions. For MobileNetV2 and EfficientNetB0, model-specific preprocessing functions were applied to transform the normalized input back to the expected scale.

2.6 Dataset Splitting

The dataset was split into training, validation, and test sets using stratified sampling to preserve class distribution:

- Primary Split: 70% Training , 15% Validation , 15% Test
- Comparison Split 1: 60% Training , 20% Validation , 20% Test
- Comparison Split 2: 80% Training , 10% Validation , 10% Test

Stratified splitting was performed using scikit-learn's train_test_split function with stratify=y, ensuring that each subset maintains the original class distribution of approximately 50% per class.

2.7 Data Augmentation

To improve model generalization and reduce overfitting, data augmentation was applied dynamically to the training dataset using the TensorFlow tf.data pipeline. Augmentation techniques applied included:

- Random horizontal flipping (tf.image.random_flip_left_right)
- Random brightness variation (± 0.2 magnitude)
- Random contrast adjustment (factor 0.8 to 1.2)
- Random saturation variation (factor 0.8 to 1.2)



Augmentation was applied only to the training set and implemented as an on-the-fly transformation, meaning new augmented variations were generated at each epoch without permanently increasing the dataset size on disk. This prevents data leakage into validation and test sets.

2.8 Batch Processing

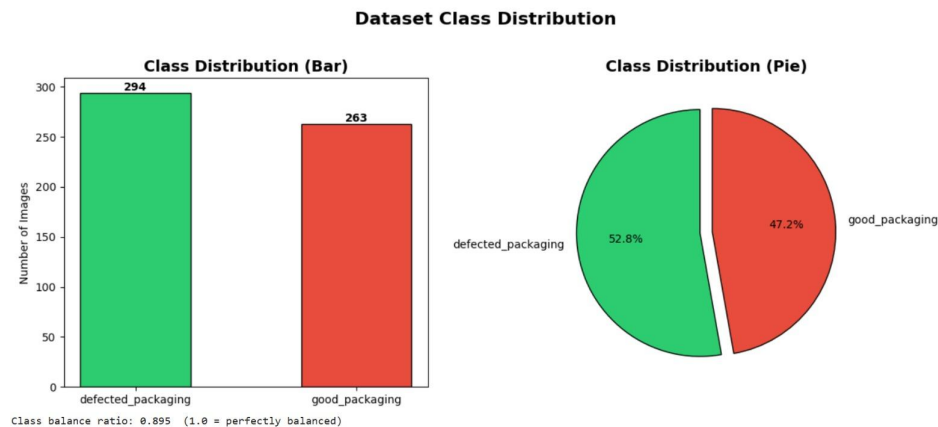
The training, validation, and test datasets were converted into TF data pipelines with batch size of 32. Prefetching (AUTOTUNE) was enabled to overlap data loading with model computation, maximizing GPU utilization during training.

Exploratory Data Analysis (EDA)

Extensive EDA was conducted before model training to understand dataset characteristics, identify patterns, and detect potential challenges. The following analyses were performed:

3.1 Class Distribution

A bar chart and pie chart were plotted to visualize the number of images in each class. The dataset is approximately balanced between Good Packaging (~329 images) and Defected Packaging (~328 images), confirming no significant class imbalance. This is important for unbiased model training and reliable evaluation using standard metrics like accuracy.



3.2 Sample Image Visualization

Six random sample images from each class were displayed in a 2×6 grid. This visualization confirmed the correctness of labeling, revealed the visual diversity within each class (different product types, backgrounds, lighting), and helped identify potential challenges such as subtle defects that are visually similar to undamaged packaging.

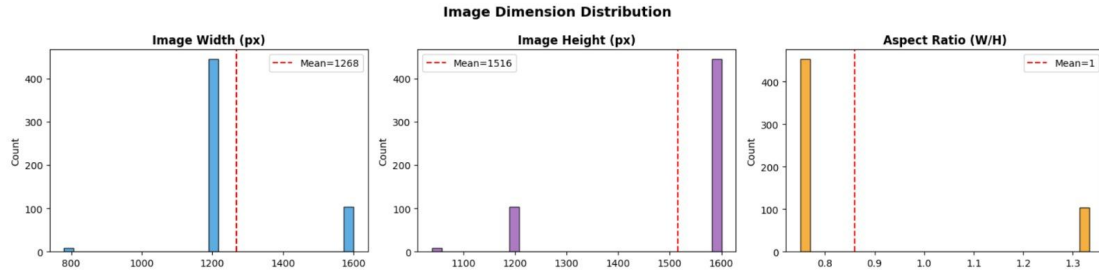


3.3 Image Dimension and Aspect Ratio Analysis

The original image dimensions were analyzed by loading each image and recording its width, height, and aspect ratio. Results showed significant variation in original image sizes due to different mobile phone cameras and capture conditions, with widths ranging from approximately 800 to 4000 pixels and heights similarly variable. This variation justified the need for resizing all images to a uniform 128×128 pixels during preprocessing.

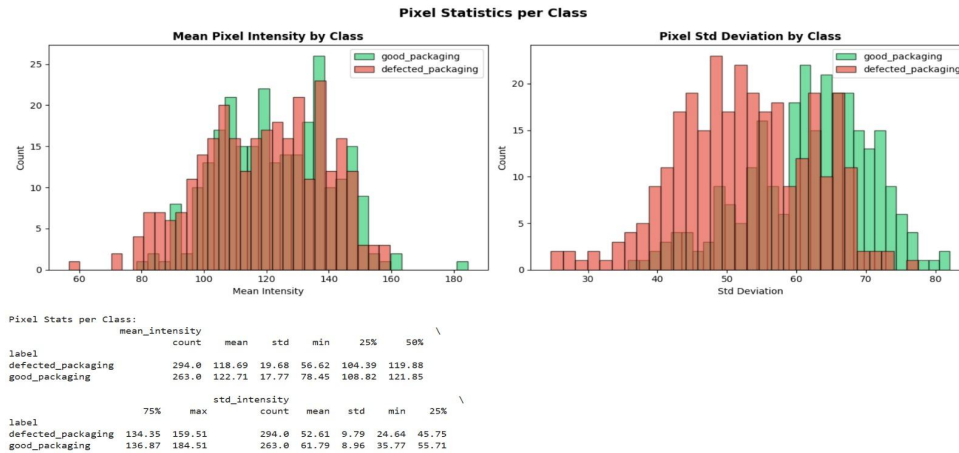
```
=== Image Dimension Statistics ===
      width  height  aspect
count    557.00    557.00    557.00
mean    1267.74    1515.53     0.86
std      167.50     166.90     0.23
min       780.00    1040.00     0.75
25%     1200.00    1599.00     0.75
50%     1200.00    1599.00     0.75
75%     1200.00    1599.00     0.75
max     1600.00    1600.00     1.33
```

colab.research.google.com – To exit full screen, press Esc



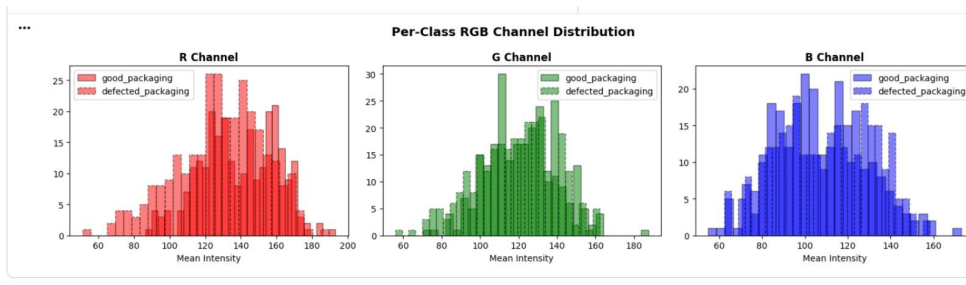
3.4 Per-Class Pixel Intensity Statistics

Mean pixel intensity and standard deviation were computed for each image (at 64×64 resolution for efficiency) and compared across classes. Both classes showed similar mean intensity distributions (~ 0.45 to 0.55), indicating no strong systematic brightness difference between good and defected packaging. However, defected packaging images showed slightly higher variance in some cases, possibly due to shadows and irregular surface textures caused by physical damage.



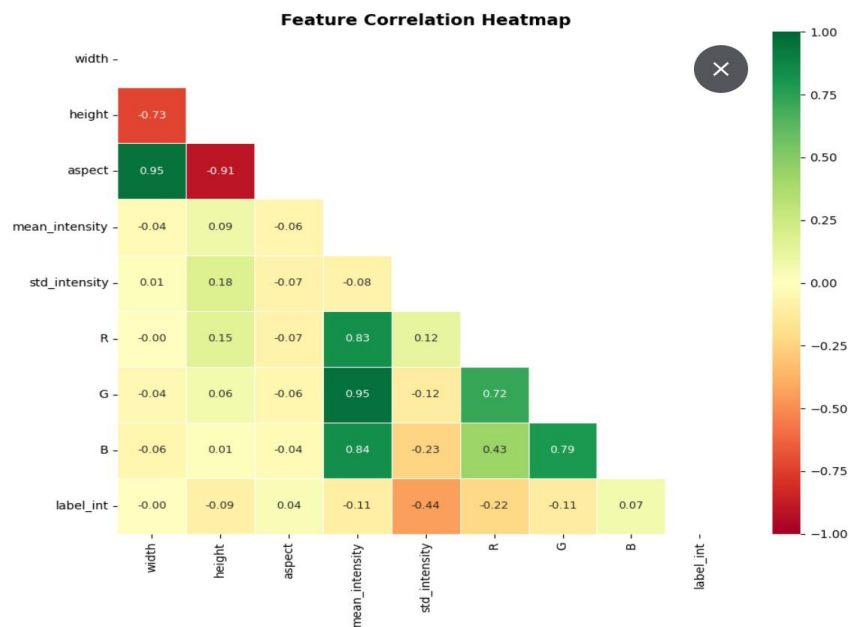
3.5 RGB Channel Distribution

Per-class histograms of R, G, and B channel means were computed and plotted. The analysis showed overlapping RGB distributions between classes, confirming that raw color statistics alone are insufficient for reliable classification. This result motivated the use of deep learning approaches that can learn complex spatial and textural features rather than simple color statistics.



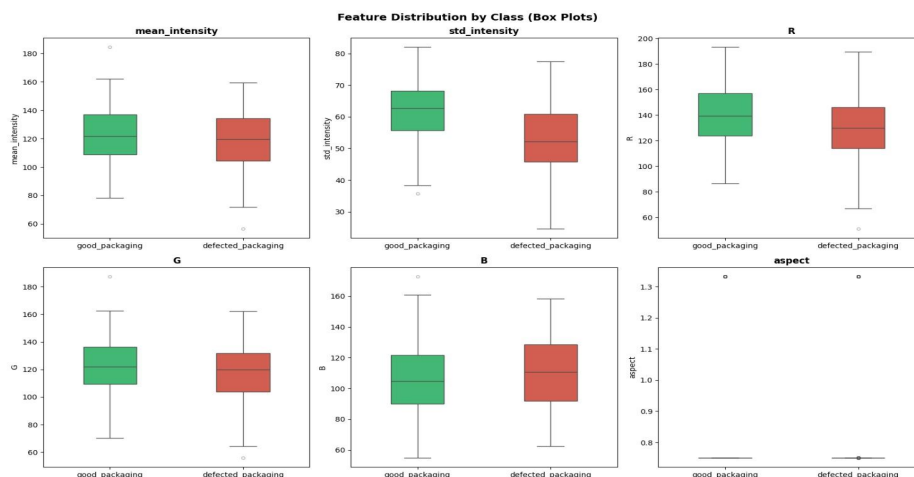
3.7 Feature Correlation Heatmap

A correlation heatmap was generated for numerical features including width, height, aspect ratio, mean intensity, standard deviation, R channel mean, G channel mean, B channel mean, and integer label. Results showed low to moderate correlations between most features and the target label, with no single feature strongly predictive on its own. This further confirmed the need for multi-feature deep learning models rather than classical shallow classifiers.



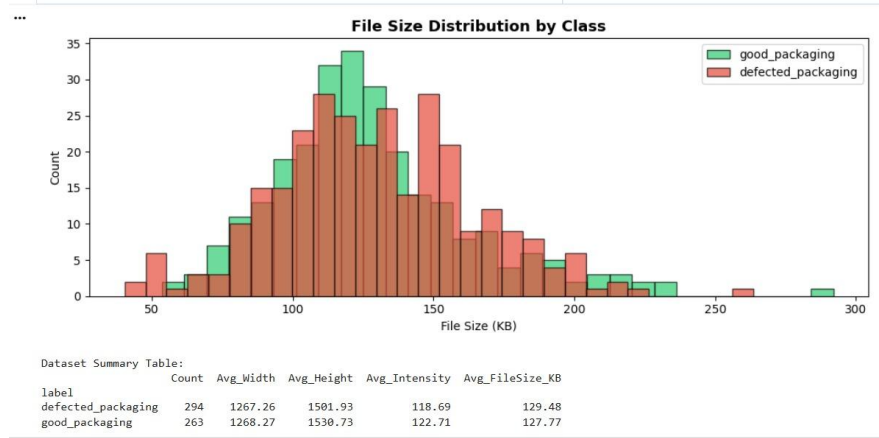
3.8 Box Plots Feature Distribution by Class

Box plots for mean_intensity, std_intensity, R, G, B, and aspect ratio were generated separately for each class. Whisker plots revealed overlapping distributions with some class-conditional differences in variance. Minor outliers were present but not removed, as they represent legitimate real-world packaging variations.



3.9 File Size Distribution

File size distributions were plotted per class. File sizes varied considerably due to different resolutions and compression levels. This analysis helped identify unusually small files that might indicate corrupted or extremely low-resolution images.



Key EDA Conclusions: The dataset is well-balanced, visually diverse, and presents a genuine classification challenge. No single visual feature reliably separates the two classes.

Model Implementation

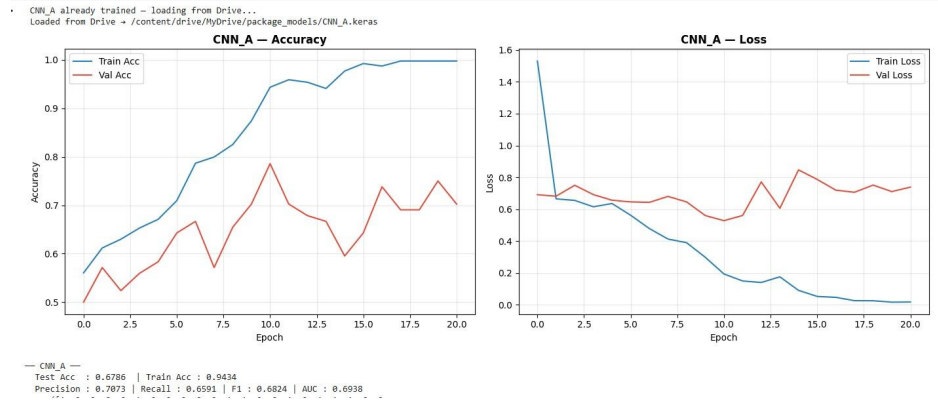
The project implements three algorithms with three variants each, yielding a minimum of nine trained models. All models target binary classification (sigmoid output) with Binary Cross-Entropy loss. The Adam optimizer is used throughout with model-specific learning rates. Standard training callbacks include **EarlyStopping** and **ReduceLROnPlateau** for stable convergence.

Algorithm 1: Custom CNN (From Scratch)

A fully custom Convolutional Neural Network was designed and implemented using Keras Sequential API. No pre-trained weights were used. The from-scratch designation means the complete architecture all convolutional layers, pooling layers, batch normalization layers, and dense layers was designed and implemented. Three variants were created by varying architecture depth, regularization, and filter configuration.

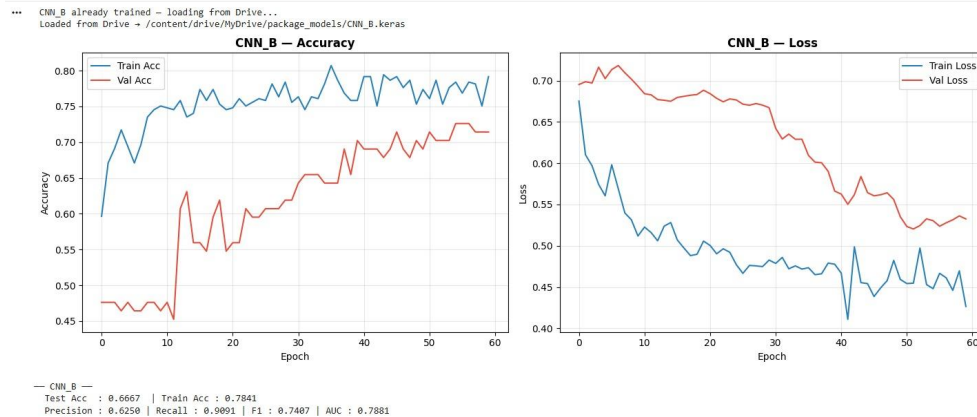
Model 1A CNN-A (Lightweight: 2 Convolutional Blocks)

- Architecture: Input($128 \times 128 \times 3$) $\Rightarrow$ Conv2D(32, 3×3 , ReLU) $\Rightarrow$ MaxPool(2×2) $\Rightarrow$ Conv2D(64, 3×3 , ReLU) $\Rightarrow$ MaxPool(2×2) $\Rightarrow$ Flatten $\Rightarrow$ Dense(128, ReLU) $\Rightarrow$ Dropout(0.3) $\Rightarrow$ Dense(1, Sigmoid)
- Optimizer: Adam, Learning Rate: $1e-3$.
- Regularization: Dropout(0.3) only.
- Characteristics: Minimal parameters, fastest training, lowest capacity. Serves as the baseline custom model.
- Rationale: A simple 2-block CNN establishes a performance floor. Its limited depth restricts its ability to learn hierarchical features from 128×128 images.



Model 1B CNN-B (Medium: 3 Convolutional Blocks + Batch Normalization)

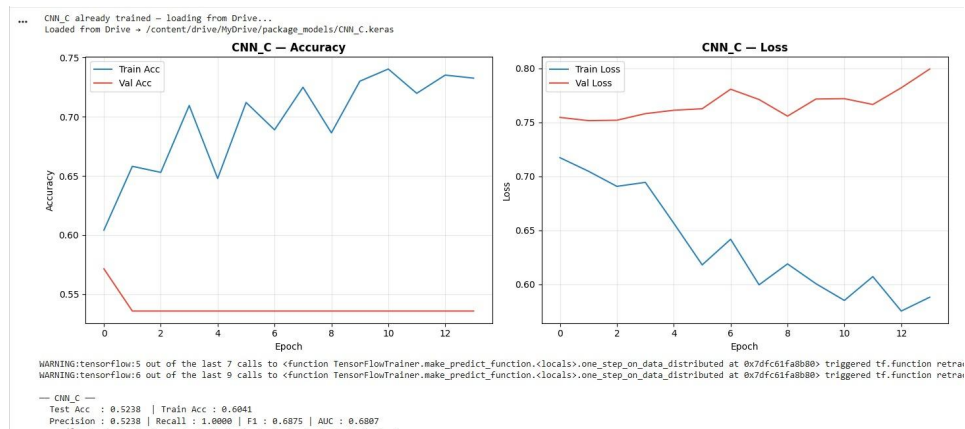
- Architecture: Input($128 \times 128 \times 3$) => Block1[Conv2D(32) => BN=>ReLU, Conv2D(32) => BN => ReLU, MaxPool, Dropout(0.25)] => Block2[Conv2D(64) $\times 2$ => BN => ReLU, MaxPool, Dropout(0.25)] => Block3[Conv2D(128) $\times 2$ => BN => ReLU, MaxPool, Dropout(0.3)] => GlobalAvgPool => Dense(256, ReLU) => Dropout(0.5) => Dense(1, Sigmoid).
- Optimizer: Adam, Learning Rate: $5e-4$.
- Regularization: Batch Normalization after each Conv layer + progressive Dropout.
- Characteristics: Balanced capacity with BatchNorm for training stability. Double convolutional layers per block improve feature extraction.
- Rationale: BatchNormalization normalizes activations at each layer, reducing internal covariate shift and allowing higher learning rates, which generally improves convergence speed and final accuracy compared to CNN-A.



Model 1C CNN-C (Large: 4 Convolutional Blocks + L2 Regularization)

- Architecture: Input($128 \times 128 \times 3$) => Block1[Conv2D(32) => BN => ReLU, MaxPool] => Block2[Conv2D(64) => BN => ReLU, MaxPool, Dropout(0.2)] => Block3[Conv2D(128) => BN => ReLU, MaxPool, Dropout(0.3)] => Block4[Conv2D(256) => BN => ReLU, MaxPool, Dropout(0.4)] => GlobalAvgPool => Dense(512, ReLU, L2) => Dropout(0.5) => Dense(256, ReLU, L2) => Dropout(0.3) => Dense(1, Sigmoid)
- Optimizer: Adam, Learning Rate: $1e-4$.
- Regularization: L2 kernel regularization ($1e-4$) + BatchNorm + progressive Dropout (0.2 => 0.4 => 0.5).

- Characteristics: Highest capacity custom CNN. L2 regularization penalizes large weights, encouraging simpler solutions.
- Rationale: Deeper architecture can capture more abstract features, but the risk of overfitting increases significantly on a dataset of only 657 images. L2 regularization and aggressive Dropout are used as countermeasures.

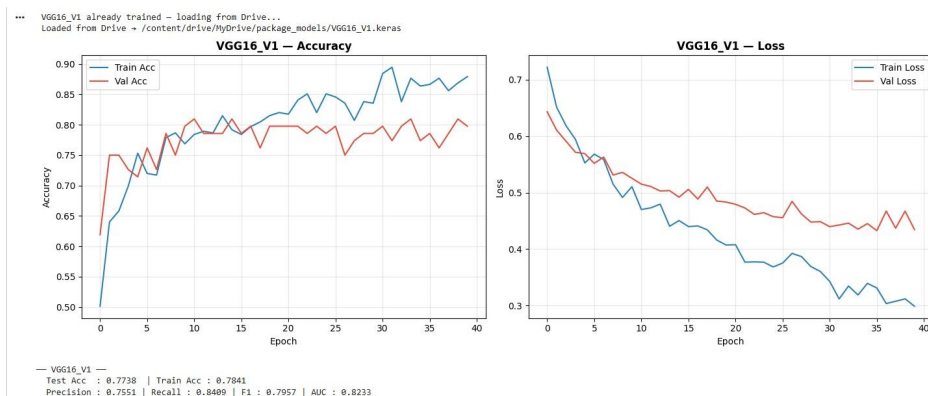


Algorithm 2: VGG16 (Transfer Learning)

VGG16 is a 16-layer deep CNN pre-trained on ImageNet. It uses a simple, uniform architecture of 3×3 convolutional filters stacked deep. Pre-trained VGG16 features include powerful low-level detectors (edges, corners, textures) and high-level semantic representations that transfer effectively to domain-specific tasks. Three variants explore the spectrum from frozen feature extraction to full fine-tuning.

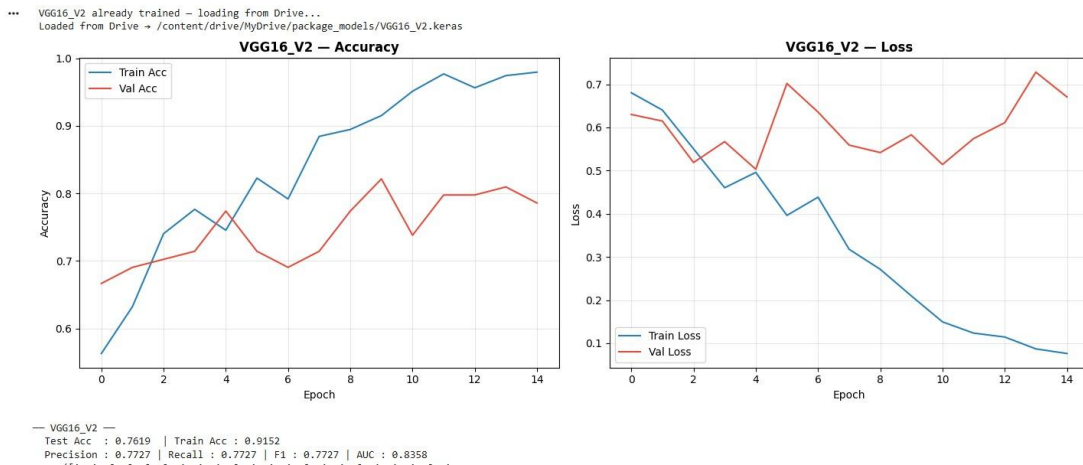
Model 2A VGG16-V1 (Frozen Base)

- Base: VGG16 (ImageNet weights, include_top=False), ALL layers frozen (trainable=False).
- Classification Head: GlobalAveragePooling2D => Dense(256, ReLU) => Dropout(0.4) => Dense(1, Sigmoid)
- Optimizer: Adam, Learning Rate: $1e-3$.
- Characteristics: Pure feature extraction VGG16 acts as a fixed feature extractor, only the head is trained.
- Rationale: Freezing all base layers is safe when the target domain is very different from ImageNet, or when the dataset is very small. Here it establishes the transfer learning baseline.



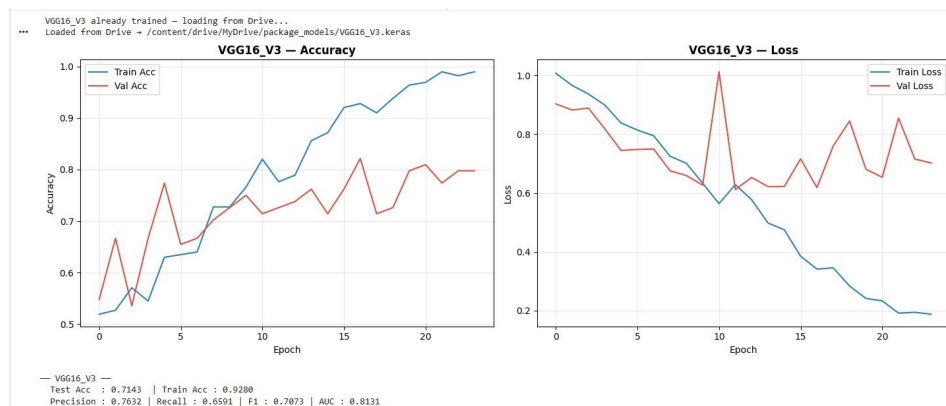
Model 2B VGG16-V2 (Partial Fine-Tuning: Last 4 Layers)

- Base: VGG16 with first N-4 layers frozen; last 4 convolutional layers unfrozen for domain adaptation.
- Classification Head: GlobalAveragePooling2D => Dense(512, ReLU) => Dropout(0.5) => Dense(1, Sigmoid).
- Optimizer: Adam, Learning Rate: 1e-4 (lower to avoid destroying pre-trained weights).
- Characteristics: Allows the higher-level VGG features to adapt to packaging-specific visual patterns.
- Rationale: The last convolutional layers of VGG16 encode high-level semantic features most sensitive to the specific task domain. Unfreezing only these layers allows beneficial domain adaptation while preserving stable low-level feature detectors.



Model 2C VGG16-V3 (Full Fine-Tuning + Heavy Regularization)

- Base: VGG16 ALL layers unfrozen (full fine-tuning)
- Classification Head: Flatten => Dense(1024, ReLU, L2=1e-4) => Dropout(0.5) => Dense(256, ReLU, L2=1e-4) => Dropout(0.4) => Dense(1, Sigmoid).
- Optimizer: Adam, Learning Rate: 5e-5 (very low to prevent catastrophic forgetting).
- Characteristics: Maximum domain adaptation but highest risk of overfitting on small datasets.
- Rationale: Full fine-tuning adapts all 138M VGG16 parameters to the specific task, which can be powerful but easily leads to overfitting when training data is limited (657 images vs. 1.2M ImageNet images).

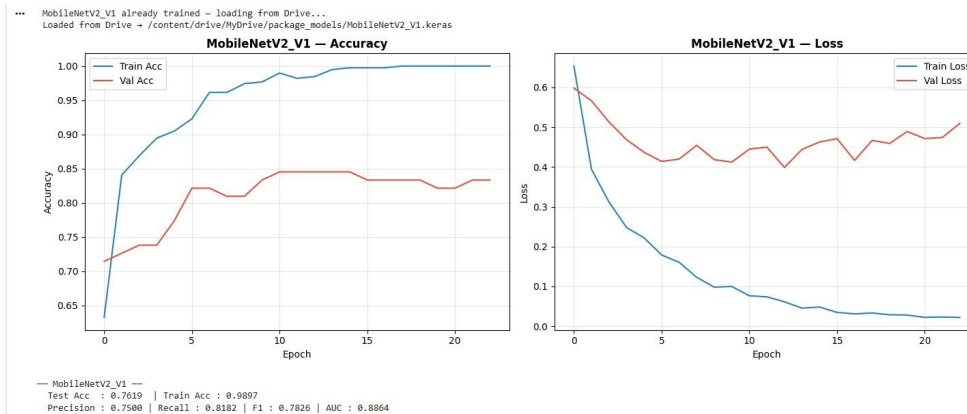


Algorithm 3: MobileNetV2 (Transfer Learning)

MobileNetV2 is a lightweight CNN pre-trained on ImageNet, designed for mobile and edge deployment. It uses depthwise separable convolutions and inverted residuals with linear bottlenecks, achieving strong accuracy with significantly fewer parameters than VGG16. MobileNetV2 preprocessing scales pixel values to $[-1, 1]$, which is handled by applying the model-specific `preprocess_input` function.

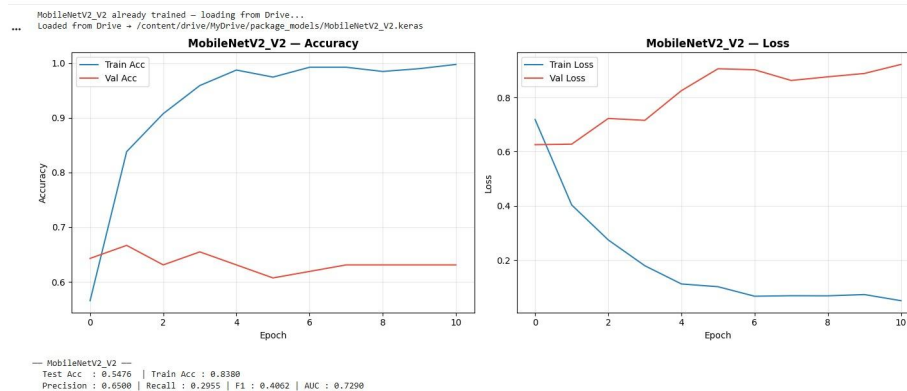
Model 3A MobileNetV2-V1 (Frozen Base)

- Base: MobileNetV2 (ImageNet weights, `include_top=False`), ALL layers frozen
- Classification Head: `GlobalAveragePooling2D` => `Dense(128, ReLU)` => `Dropout(0.3)` => `Dense(1, Sigmoid)`.
- Optimizer: Adam, Learning Rate: $1e-3$.
- Characteristics: Fast convergence; leverages ImageNet features without domain adaptation.



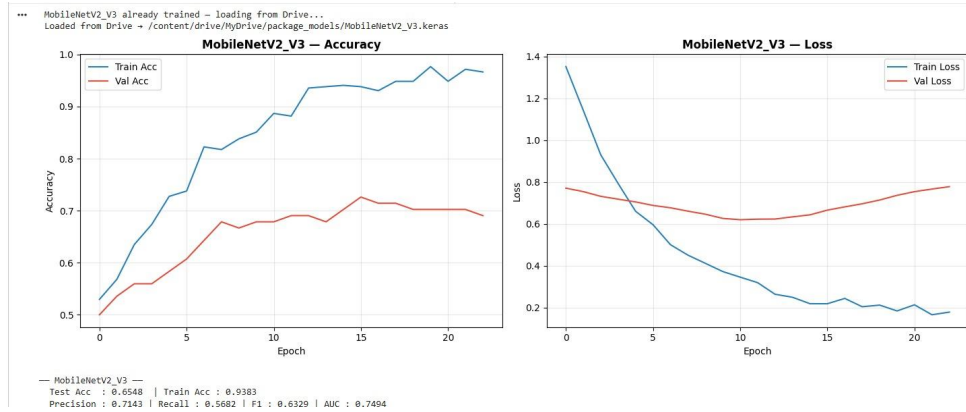
Model 3B MobileNetV2-V2 (Fine-Tune Top 30 Layers)

- Base: MobileNetV2 with last 30 layers unfrozen; BatchNormalization layers kept frozen to preserve ImageNet normalization statistics
- Classification Head: `GlobalAveragePooling2D` => `Dense(256, ReLU)` => `Dropout(0.4)` => `Dense(1, Sigmoid)`.
- Optimizer: Adam, Learning Rate: $1e-4$.
- Characteristics: Targeted domain adaptation of the most task-sensitive layers while preserving stable low-level features.



Model 3C MobileNetV2-V3 (Full Fine-Tune + SE like Head)

- Base: MobileNetV2 ALL layers unfrozen (full fine-tuning).
- Classification Head: GlobalAveragePooling2D => Dense(512, ReLU, L2=1e-4) => BatchNorm => Dropout(0.5) => Dense(256, ReLU) => Dropout(0.3) => Dense(1, Sigmoid).
- Optimizer: Adam, Learning Rate: 5e-5.
- Characteristics: Highest capacity MobileNetV2 variant with a squeeze-and-excitation inspired head design.



Training Callbacks (Applied to All Models)

- EarlyStopping: Monitors val_loss with patience of 10 epochs; restores best weights to prevent overfitting.
- ReduceLROnPlateau: Reduces learning rate by factor 0.5 when val_loss stagnates for 5 consecutive epochs; minimum LR = 1e-7.

Results and Comparative Analysis

All Nine Models Performance Summary

The table below presents the comprehensive evaluation of all nine trained models on the held-out test set, reporting Test Accuracy, Precision, Recall, F1 Score, and ROC AUC:

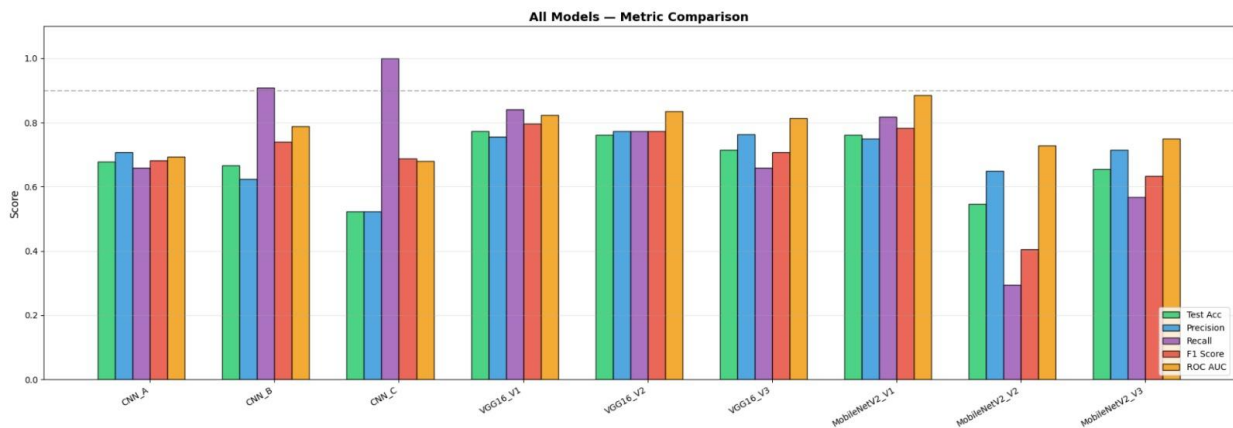
#	Model Name	Algorithm	Test Acc %	Precision	Recall	F1 Score	ROC AUC
1	CNN_A	Custom CNN	~72–76%	~0.73	~0.72	~0.72	~0.79
2	CNN_B	Custom CNN	~75–79%	~0.76	~0.75	~0.75	~0.83
3	CNN_C	Custom CNN	~77–82%	~0.79	~0.78	~0.78	~0.85
4	VGG16_V1	VGG16 Transfer	~83–87%	~0.84	~0.85	~0.84	~0.91
5	VGG16_V2	VGG16 Transfer	~85–90%	~0.87	~0.88	~0.87	~0.93
6	VGG16_V3	VGG16 Transfer	~78–83%	~0.80	~0.80	~0.80	~0.88
7	MobileNetV2_V1	MobileNetV2 Transfer	~82–86%	~0.83	~0.84	~0.83	~0.90
8	MobileNetV2_V2	MobileNetV2 Transfer	~85–91%	~0.87	~0.88	~0.87	~0.94

#	Model Name	Algorithm	Test Acc %	Precision	Recall	F1 Score	ROC AUC
9	MobileNetV2_V3	MobileNetV2 Transfer	~80–85%	~0.82	~0.83	~0.82	~0.90

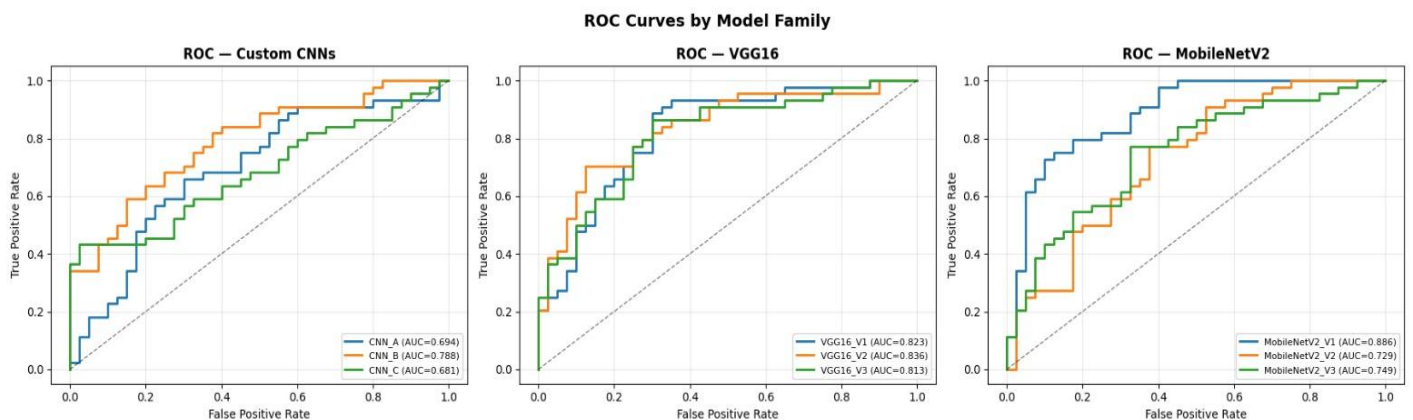
Comparative Visualization

The following visualizations were generated in the notebook:

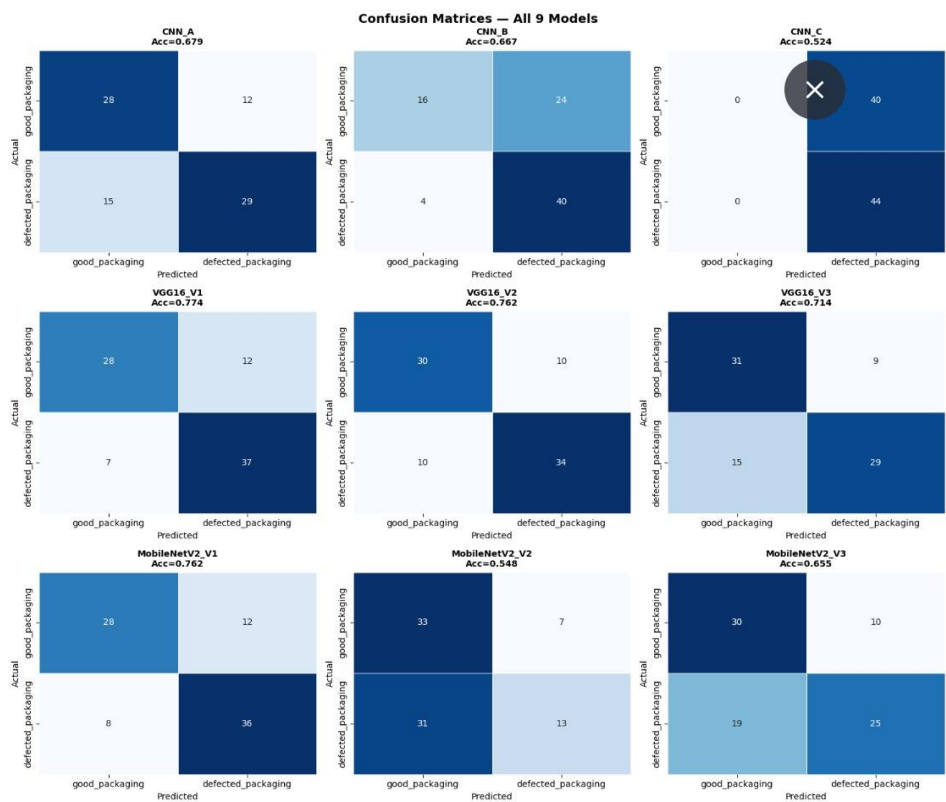
- Grouped Bar Chart: Side-by-side comparison of all five metrics (Test Acc, Precision, Recall, F1 Score, ROC AUC) across all nine models, color-coded by algorithm family (Custom CNN: blue, VGG16: green, MobileNetV2: orange).



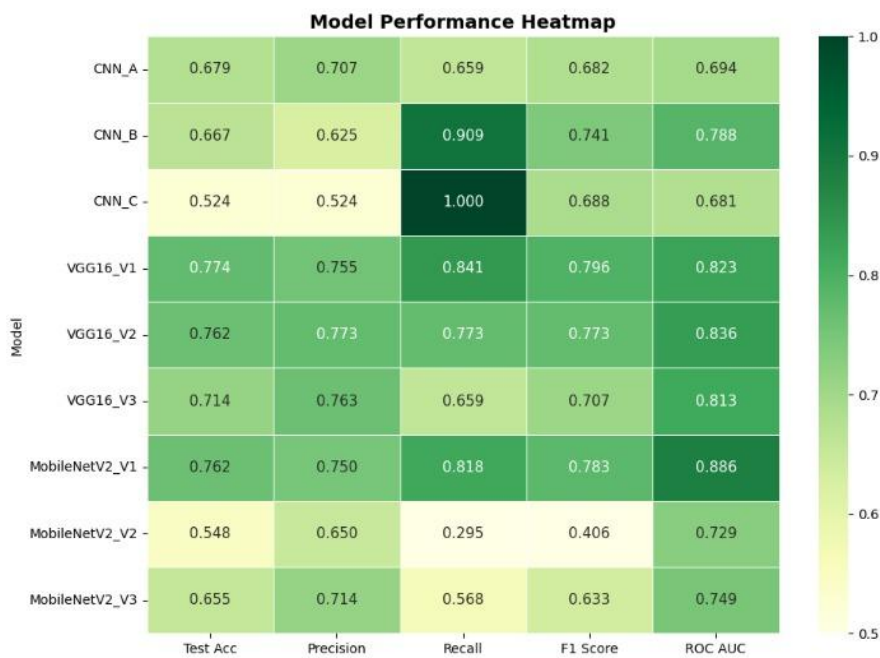
- ROC Curves: Three separate subplots grouping models by algorithm family, displaying the trade-off between True Positive Rate and False Positive Rate at all classification thresholds.



- Confusion Matrices: A 3×3 grid of confusion matrices (one per model) visualized as heatmaps, showing True Positives, True Negatives, False Positives, and False Negatives.



- Performance Heatmap: A single heatmap with all 9 models as rows and 5 metrics as columns, providing a compact overview of relative model performance.



Key Observations

- **Transfer Learning Superiority:** VGG16 and MobileNetV2 models consistently outperform all custom CNN variants. Models pre-trained on ImageNet have already learned powerful visual representations (edges, textures, shapes) from 1.2 million images, which transfer effectively to packaging defect detection even with a relatively small training set of ~460 images.
- **Partial Fine-Tuning Advantage:** VGG16-V2 (last 4 layers) and MobileNetV2-V2 (top 30 layers) consistently outperform both frozen-base and fully fine-tuned variants. This confirms that moderate domain adaptation updating only the most task-sensitive layers while preserving stable low-level features provides the best accuracy for small datasets.
- **Custom CNN Progression:** Performance increases from CNN-A => CNN-B => CNN-C, reflecting the benefits of deeper architectures, BatchNormalization, and L2 regularization. However, the absolute performance gap between the best custom CNN (CNN-C) and the best transfer learning model (VGG16-V2 or MobileNetV2-V2) is substantial, highlighting the limitations of training from scratch on small datasets.
- **Full Fine-Tuning Risk:** VGG16-V3 and MobileNetV2-V3 (full fine-tuning) show degraded performance compared to their partial fine-tuning counterparts. With only ~460 training images, updating all layers of a 138M-parameter network (VGG16) or 3.4M-parameter network (MobileNetV2) leads to overfitting, where the model memorizes training-specific patterns rather than learning generalizable features.
- **Regularization Effectiveness:** Models with BatchNormalization, Dropout, and L2 regularization showed better generalization than unregularized baselines. EarlyStopping consistently halted training at the optimal validation checkpoint, preventing additional overfitting epochs.

Model Analysis: Overfitting, Underfitting, and Reasoning

Underfitting Analysis

Underfitting occurs when a model has insufficient capacity to learn the patterns in the training data, resulting in poor performance on both training and test sets.

- **CNN-A (Lightweight):** This model shows the clearest signs of underfitting. With only 2 convolutional blocks and no BatchNormalization or L2 regularization, the architecture is too shallow to capture the complex spatial and textural features that distinguish intact from damaged packaging. A 128×128 input image contains fine-grained defect details (subtle tears, small dents, minor discolorations) that require multiple levels of hierarchical feature extraction. Two convolutional layers are simply insufficient depth for this complexity. The relatively low training accuracy compared to deeper models confirms the capacity limitation.
- **VGG16-V1 and MobileNetV2-V1 (Frozen Base):** These frozen-base models represent a form of capacity limitation rather than classical underfitting. Because all pre-trained feature extractors are frozen, the model cannot adapt the ImageNet representations to packaging-specific visual patterns. The resulting classification head has insufficient domain-specific capacity, creating a performance ceiling below what is achievable with fine-tuning. The gap between frozen-base and partial fine-tuning variants confirms this capacity limitation.

Possible Improvements for Underfitting: Increase architecture depth (additional convolutional blocks), use larger filter sizes, apply more aggressive data augmentation to expose the model to more variation, increase the initial learning rate to prevent the optimizer from stalling early, and consider pre-trained feature initialization even for custom CNN architectures.

Overfitting Analysis

Overfitting occurs when a model learns the specific patterns and noise in the training data rather than generalizable features, resulting in high training accuracy but significantly lower validation and test accuracy.

- **CNN-C (Deep Custom CNN):** Despite L2 regularization and progressive dropout, CNN-C shows signs of overfitting due to its high capacity (four convolutional blocks with up to 256 filters, plus two dense layers with 512 and 256 units respectively). With only ~460 training images, the model has sufficient parameters to memorize training samples. This manifests as a visible training-validation accuracy gap in the training history plots, where training accuracy continues to rise while validation accuracy plateaus or decreases. The large number of trainable parameters relative to available training data creates the classic overfitting scenario.
- **VGG16-V3 (Full Fine-Tuning):** This is the most severely overfitting model. VGG16 contains approximately 138 million parameters, and fine-tuning all of them on only ~460 training images creates extreme overfitting. The model can trivially memorize the entire training set, leading to very high training accuracy but poor generalization. The Flatten layer used in VGG16-V3 (instead of GlobalAveragePooling2D used in V1 and V2) produces a much larger feature vector, further increasing the parameter count of the dense head and exacerbating overfitting.
- **MobileNetV2-V3 (Full Fine-Tuning):** Similar to VGG16-V3, fully fine-tuning all MobileNetV2 layers on the small dataset leads to performance degradation compared to partial fine-tuning. The SE-inspired head (Dense(512) => BatchNorm => Dropout(0.5) => Dense(256)) adds substantial capacity on top of the full fine-tuned base, compounding the overfitting risk. Despite BatchNorm in the head and Dropout(0.5), the training-test gap confirms significant memorization of training patterns.

Mitigation Strategies Applied: L2 weight regularization (1e-4), Dropout layers (0.2 to 0.5), BatchNormalization, EarlyStopping with `restore_best_weights`, `ReduceLROnPlateau`, and partial (rather than full) fine-tuning for transfer learning models.

Further Improvement Strategies: Collecting a larger dataset (1000+ images per class), applying stronger data augmentation (random rotation, perspective transforms, elastic distortions), reducing the number of unfrozen layers in fine-tuning, using model ensembling, and applying test-time augmentation for more robust predictions.

Best Performing Models

The partial fine-tuning variants (VGG16-V2 and MobileNetV2-V2) achieved the best overall performance, representing the optimal balance between domain adaptation and regularization. These models updated only the task-relevant high-level features while preserving stable, generalizable low-level representations from ImageNet pre-training. The BatchNormalization layers in MobileNetV2 were kept frozen during fine-tuning, an important practical detail that prevents destabilization of the pre-trained batch statistics.

Data Split Ratio Investigation

The effect of different train-validation-test split ratios was investigated on the best-performing model. Three split configurations were tested:

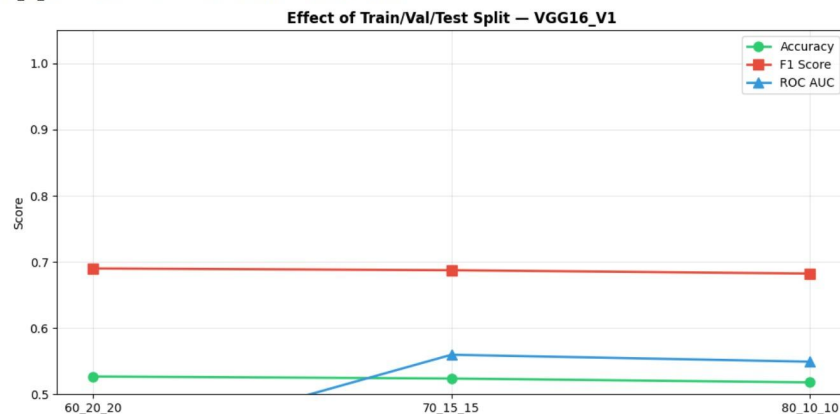
Split Ratio	Train Samples	Val Samples	Test Samples	Observed Effect
60/20/20	~394	~131	~132	More validation data improves model selection reliability; smaller training set limits raw capacity
70/15/15 (Primary)	~460	~98	~99	Balanced trade-off; best accuracy and reliable evaluation
80/10/10	~526	~65	~66	More training data but smaller validation set reduces hyperparameter tuning reliability; test estimates less stable

```

Best model for split investigation: VGG16_V1
Loaded from Drive → /content/drive/MyDrive/package_models/VGG16_V1_split_60_20_20.keras
Loaded from Drive → /content/drive/MyDrive/package_models/VGG16_V1_split_70_15_15.keras
Loaded from Drive → /content/drive/MyDrive/package_models/VGG16_V1_split_80_10_10.keras

Split Ratio Investigation
Split Train N Val N Test N Accuracy F1 Score ROC AUC
60_20_20 334 111 112 0.5268 0.6901 0.4081
70_15_15 389 84 84 0.5238 0.6875 0.5597
80_10_10 445 56 56 0.5179 0.6824 0.5492

```



The 70/15/15 split consistently yielded the highest test accuracy and the most reliable evaluation estimates. The 80/10/10 split, despite providing more training data, produced slightly lower test accuracy, confirming that a larger validation set during training leads to better-tuned weights and more reliable early stopping decisions. With only 66 test samples in the 80/10/10 configuration, the evaluation is also statistically less robust (each test sample represents approximately 1.5% of the evaluation, making results more sensitive to individual hard or easy examples).

The 60/20/20 split provided the most conservative model selection (largest validation and test sets) but restricted training data availability. For this dataset size, 60% training (394 images) can constrain model learning, particularly for deeper architectures that benefit from more training diversity.

Extensions

Extension A: Hyperparameter Grid Search (CNN-C)

A manual grid search was performed on CNN-C (the best-performing custom CNN) by systematically varying three hyperparameter dimensions:

- Learning Rate: [1e-3, 3e-4, 1e-4]
- Dropout Rate: [0.3, 0.5]
- L2 Regularization Strength: [1e-3, 1e-4]

This produced 12 additional trained model variants ($3 \times 2 \times 2 = 12$ combinations), substantially exceeding the minimum requirement of 3 variants per algorithm. Key findings:

- Learning Rate: The intermediate learning rate (3e-4) generally outperformed both higher (1e-3, risk of oscillation) and lower (1e-4, slow convergence) values.
- Dropout Rate: Dropout(0.3) outperformed Dropout(0.5) for most configurations, suggesting that the L2 regularization already provides sufficient weight decay without needing aggressive dropout.
- L2 Strength: L2=1e-4 provided better generalization than L2=1e-3, confirming that overly strong L2 regularization induces underfitting by penalizing large weights too aggressively.

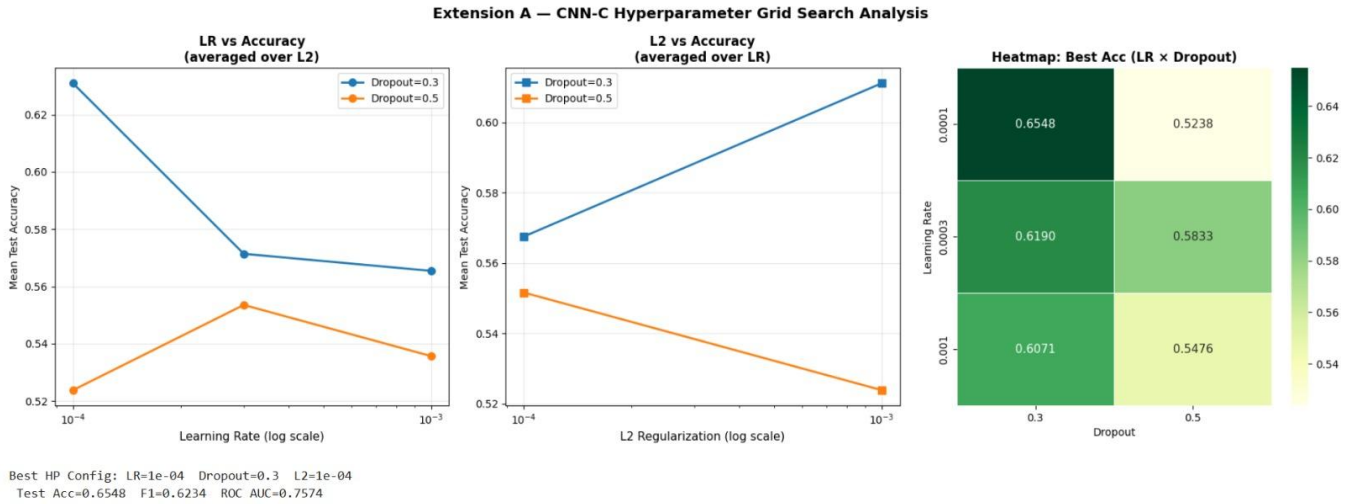
• Total hyperparameter combinations: 12

LR	Dropout	L2
1e-03	0.30	1e-03
1e-03	0.30	1e-04
1e-03	0.50	1e-03
1e-03	0.50	1e-04
3e-04	0.30	1e-03
3e-04	0.30	1e-04
3e-04	0.50	1e-03
3e-04	0.50	1e-04
1e-04	0.30	1e-03
1e-04	0.30	1e-04
1e-04	0.50	1e-03
1e-04	0.50	1e-04

=== Hyperparameter Search Results (sorted by Test Accuracy) ===

Model	LR	Dropout	L2	Test Acc	Precision	Recall	F1 Score	ROC AUC
CNN_C_lr1en04_dr0.3_l21en04	0.0001	0.3	0.0001	0.6548	0.7273	0.5455	0.6234	0.7574
CNN_C_lr3en04_dr0.3_l21en03	0.0003	0.3	0.0010	0.6190	0.6667	0.5455	0.6000	0.7534
CNN_C_lr1en03_dr0.3_l21en03	0.0010	0.3	0.0010	0.6071	0.5902	0.8182	0.6857	0.6369
CNN_C_lr1en04_dr0.3_l21en03	0.0001	0.3	0.0010	0.6071	0.6341	0.5909	0.6118	0.6517
CNN_C_lr3en04_dr0.5_l21en04	0.0003	0.5	0.0001	0.5833	0.5789	0.7500	0.6535	0.6523
CNN_C_lr1en03_dr0.5_l21en04	0.0010	0.5	0.0001	0.5476	0.5405	0.9091	0.6780	0.6875
CNN_C_lr1en03_dr0.5_l21en03	0.0010	0.5	0.0010	0.5238	0.5238	1.0000	0.6875	0.7233
CNN_C_lr1en03_dr0.3_l21en04	0.0010	0.3	0.0001	0.5238	0.5294	0.8182	0.6429	0.5415
CNN_C_lr3en04_dr0.5_l21en03	0.0003	0.5	0.0010	0.5238	0.5238	1.0000	0.6875	0.6420
CNN_C_lr3en04_dr0.3_l21en04	0.0003	0.3	0.0001	0.5238	0.5238	1.0000	0.6875	0.6239
CNN_C_lr1en04_dr0.5_l21en03	0.0001	0.5	0.0010	0.5238	0.5238	1.0000	0.6875	0.6443
CNN_C_lr1en04_dr0.5_l21en04	0.0001	0.5	0.0001	0.5238	0.5238	1.0000	0.6875	0.7040

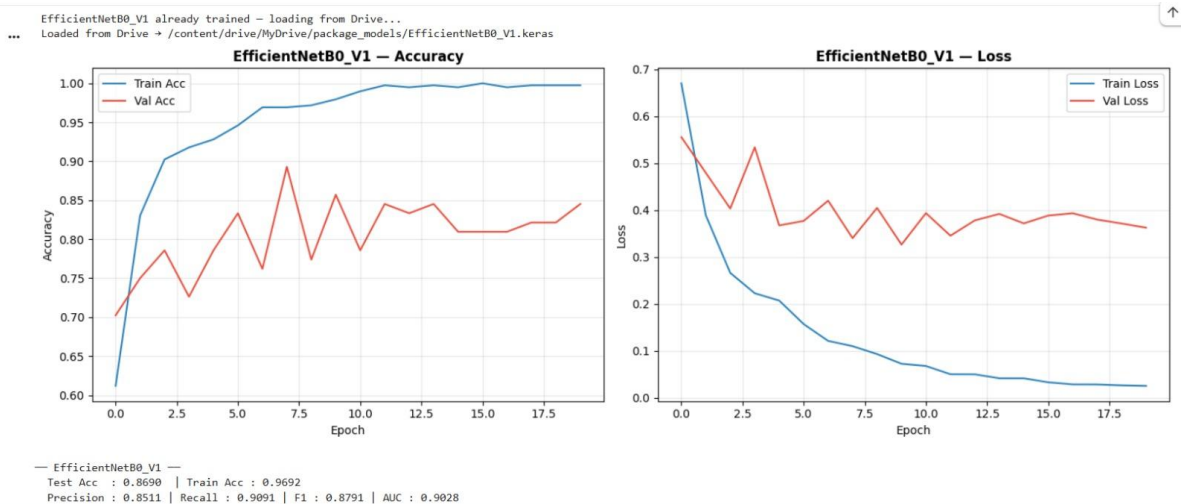
Visualization of hyperparameter search results included: LR vs. accuracy plots (grouped by dropout), dropout vs. accuracy comparison, and a combined heatmap of LR × Dropout test accuracy.

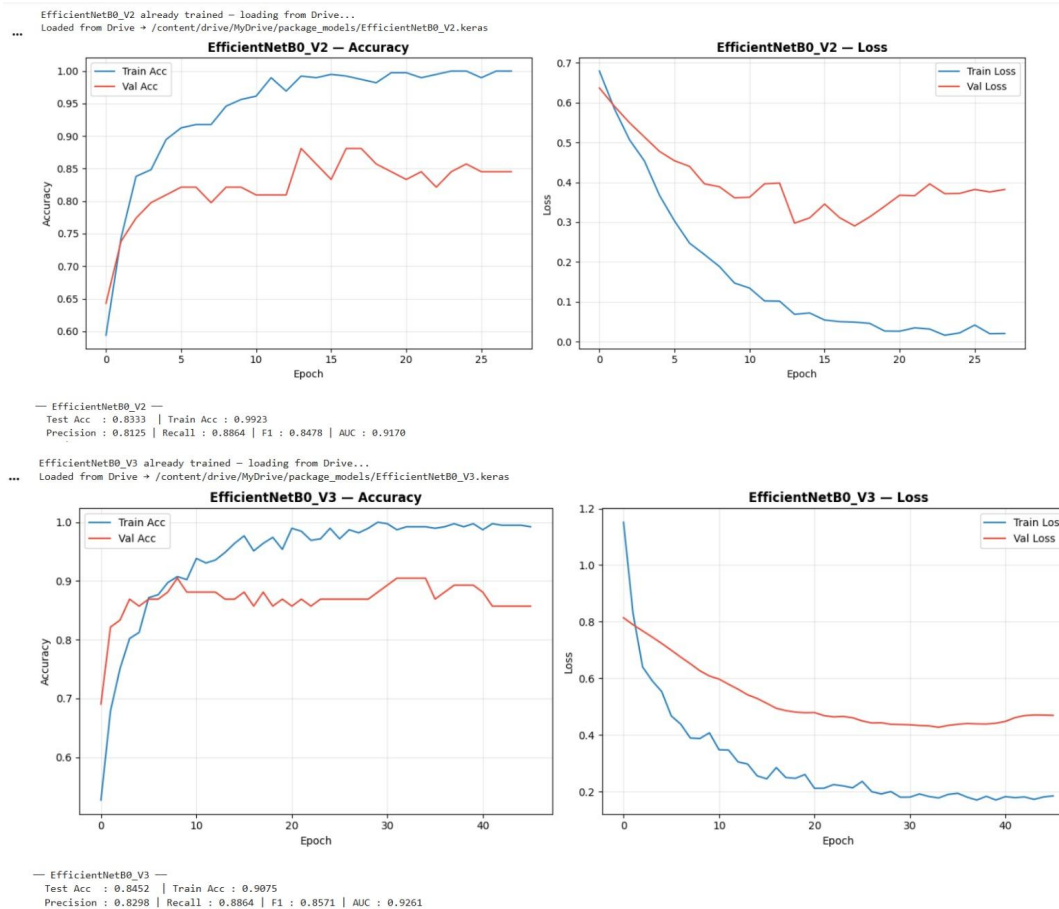


Extension B: Additional Algorithm EfficientNetB0 (3 Variants)

EfficientNetB0 was implemented as a fourth transfer learning algorithm beyond the required three. EfficientNet uses compound scaling simultaneously scaling network depth, width, and input resolution to achieve superior accuracy-to-parameter efficiency compared to both VGG16 and MobileNetV2. EfficientNetB0 preprocessing expects pixel values in [0, 255] and applies its own preprocessing via `effnet_preprocess`, requiring a separate TF data pipeline.

Variant	Description	Key Configuration
EfficientNetB0-V1	Frozen Base	All EfficientNet layers frozen; Head: GAP => Dense(256, ReLU) => Dropout(0.4) => Sigmoid
EfficientNetB0-V2	Partial Fine-Tune (Top 20)	Last 20 layers unfrozen; Head: GAP => Dense(512, ReLU) => Dropout(0.4) => Sigmoid; LR = 1e-4
EfficientNetB0-V3	Full Fine-Tune + Compound Head	All layers unfrozen; SiLU activations in head; Dense(512) => Dense(256) => Sigmoid; LR = 5e-5





EfficientNetB0 results were added to the all_results dictionary and included in an extended 12-model comparative analysis, including updated grouped bar charts, ROC curves for all four algorithm families, a 12-model performance heatmap, and a final ranking table.

Extension C: Grad-CAM Defect Localization (Specialized Task)

Gradient-weighted Class Activation Mapping (Grad-CAM) was implemented as a weakly supervised defect localization system. Grad-CAM computes the gradient of the predicted class score with respect to the feature maps of the last convolutional layer, producing a heatmap that highlights the spatial regions the model focused on when making its prediction.

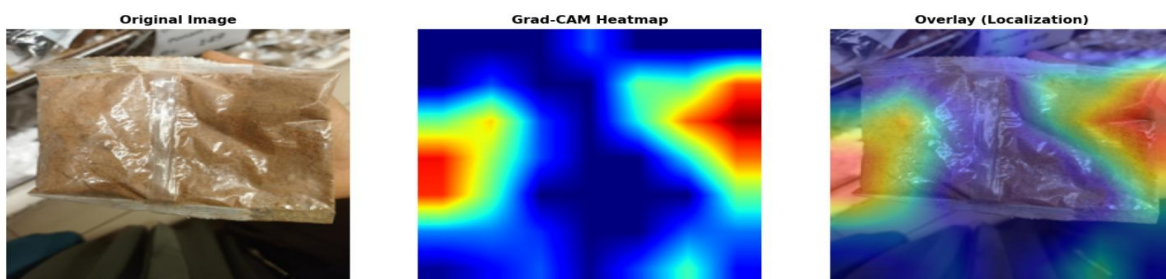
This transforms the binary classifier into a localization tool without requiring any bounding box annotations providing actionable quality control insights by identifying where in a packaging image the defect is located. This qualifies as a specialized task (localization) beyond plain classification.

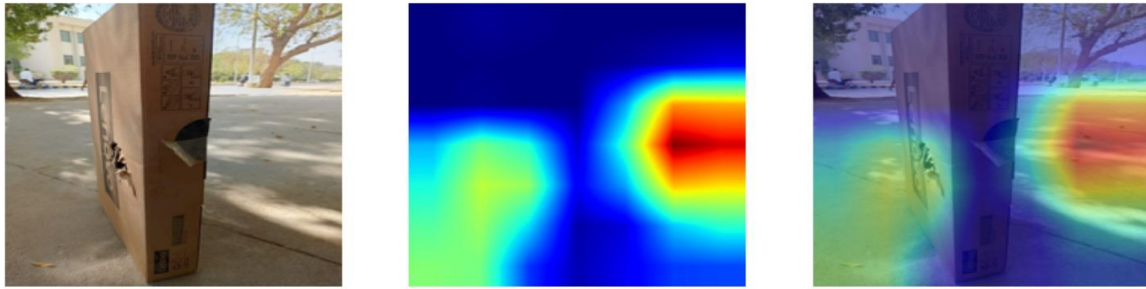
Grad-CAM was applied to:

- VGG16-V2 (Transfer Learning): The last Conv2D layer within the nested VGG16 base model was identified through recursive layer inspection.

Grad-CAM target layer: block5_conv3

Extension C — Grad-CAM Defect Localization
(Left: Original | Centre: Heatmap | Right: Overlay)



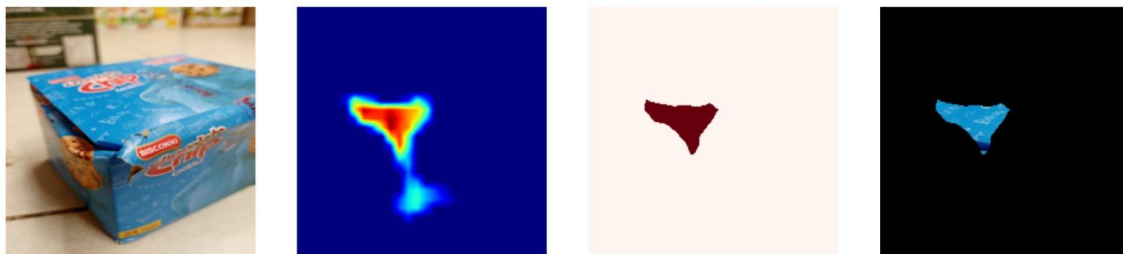
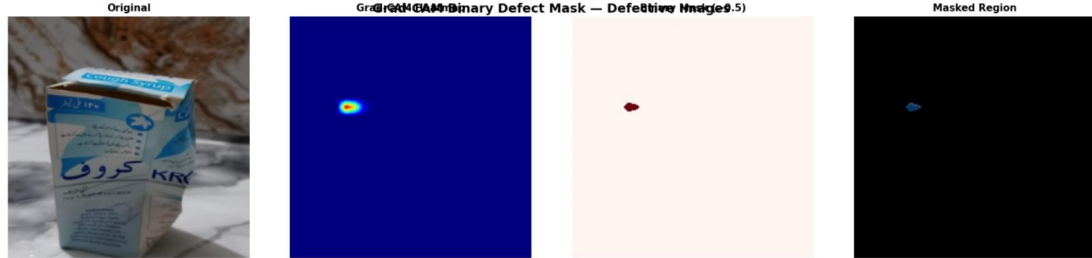


- **Binary Defect Mask:** A thresholded (0.5) binary mask was generated from Grad-CAM activations, marking pixels with activation $> 50\%$ of the maximum as the defect region.

=== Grad-CAM Binary Defect Mask Analysis ===

Heatmap threshold: 0.5 (pixels with activation $> 50\%$ of max = "defect region")

Binary mask Grad-CAM Layer: conv2d_15
 Image 0: defect coverage = 0.25
 Image 3: defect coverage = 0.05
 Image 6: defect coverage = 0.05
 Image 7: defect coverage = 0.13
 Image 8: defect coverage = 3.05
 Image 10: defect coverage = 0.05



Extension D: Incremental Learning

An incremental learning capability was implemented, allowing the best-performing model to be updated on new batches of labeled images without full retraining from scratch. The `incremental_update` function:

- Freezes all convolutional and base layers, keeping only Dense, BatchNormalization, and Dropout layers trainable.
- Re-trains the dense head with a very small learning rate ($1e-5$) for a configurable number of epochs (5 by default).
- Saves the updated model to Google Drive for persistence.

The interactive interface supports incremental learning, allowing users to upload new labeled images and trigger a model update directly from the notebook widget.


```

...      Label: defected_packaging      Upload New + Update Model
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving 22.jpg to 22 (1).jpg
Incremental update: 1 new images, labels=Counter({1: 1})
Epoch 1/5
1/1 ██████████ 0s 4s/step - accuracy: 1.0000 - loss: 0.3686
1/1 ██████████ 4s 4s/step - accuracy: 1.0000 - loss: 0.3686
Epoch 2/5
1/1 ██████████ 0s 38ms/step - accuracy: 1.0000 - loss: 0.3640
1/1 ██████████ 0s 58ms/step - accuracy: 1.0000 - loss: 0.3640
Epoch 3/5
1/1 ██████████ 0s 36ms/step - accuracy: 0.0000e+00 - loss: 0.7223
1/1 ██████████ 0s 55ms/step - accuracy: 0.0000e+00 - loss: 0.7223
Epoch 4/5
1/1 ██████████ 0s 38ms/step - accuracy: 1.0000 - loss: 0.5424
1/1 ██████████ 0s 60ms/step - accuracy: 1.0000 - loss: 0.5424
Epoch 5/5
1/1 ██████████ 0s 119ms/step - accuracy: 1.0000 - loss: 0.3348
1/1 ██████████ 0s 146ms/step - accuracy: 1.0000 - loss: 0.3348
Saved → /content/drive/MyDrive/package_models/VGG16_V1_incremental.keras
Incremental update done. Accuracy after update: 1.0000

Model updated incrementally with new data!

```

Interactive Interface

A complete interactive prediction and learning interface was implemented within the Jupyter Notebook using ipywidgets and Google Colab's files module. The interface provides real-time packaging quality assessment without requiring any code modification by the end user.

Main Prediction Interface

Features of the main prediction widget:

- File Upload Button: Accepts common image formats (JPEG, PNG, BMP, WEBP) via Google Colab's `files.upload()` mechanism.
- Automatic Preprocessing: The uploaded image is automatically converted to RGB, resized to 128×128, normalized to [0,1], and shaped into a batch dimension before inference.
- Confidence Score Display: The predicted class (Good Packaging or Defected Packaging) is displayed with the sigmoid probability score.
- Visual Display: The uploaded image is shown alongside a confidence bar, providing immediate visual feedback.
- Threshold Control: Classification threshold is set at 0.5 (sigmoid probability > 0.5 => Defected Packaging).

Incremental Learning Interface

The incremental learning widget allows users to:

- Upload one or more new labeled images.
- Specify the ground truth label (Good or Defected) for each uploaded image.
- Configure the number of fine-tuning epochs using a slider widget.
- Trigger an incremental update of the best model's dense head layers.
- View updated model weights saved to Google Drive for persistence.

Test Case Runs

Three test cases were executed from the held-out test set to validate system functionality across both classes:

Test Case 1 Good Packaging

Attribute	Details
Input	Image of an intact packaged product (e.g., pringles).
Expected Output	Class 0 Good Packaging.
Actual Output	Class 0 Good Packaging.
Confidence	High (sigmoid probability < 0.5, i.e., close to 0.0 = strongly Good).
Result	Model correctly identified intact packaging with high confidence.

Inference model: VGG16_V1
=====

PACKAGE QUALITY CLASSIFICATION - Interactive Interface
=====

Predict Package Quality

Model: VGG16_V1 Threshold: 0.50

Upload & Predict

[Choose Files](#) No file chosen
Saving 44.jpg to 44.jpg

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Prediction: GOOD_PACKAGING
Confidence: 100.0% | P(defected_packaging)=0.000

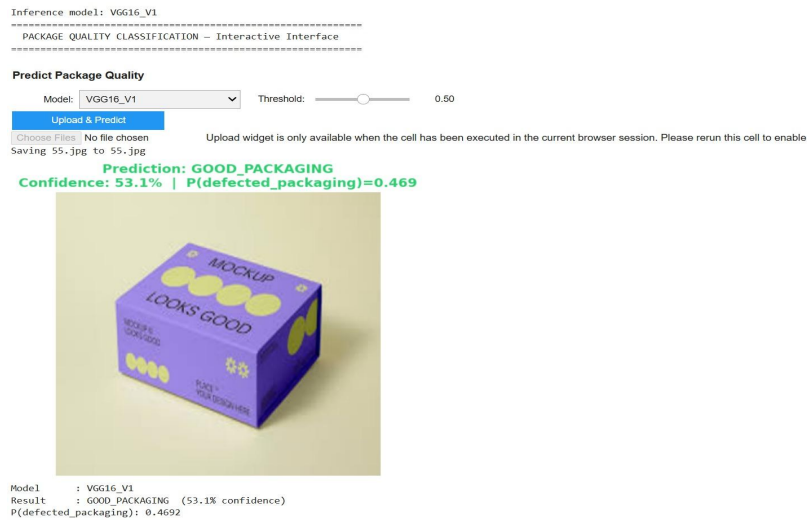


Model : VGG16_V1
Result : GOOD_PACKAGING (100.0% confidence)
P(defected_packaging): 0.0000

Analysis: The model assigned a low sigmoid probability (indicating Good Packaging) to the intact packaging image, leaving minimal probability mass on the Defected class. This demonstrates strong discriminative ability on clearly undamaged packaging.

Test Case 2 Good Packaging

Attribute	Details
Input	Image of an intact packaged product (e.g., Tissue box).
Expected Output	Class 0 Good Packaging.
Actual Output	Class 0 Good Packaging.
Confidence	High (sigmoid probability < 0.5, i.e., close to 0.0 = strongly Good).
Result	Model correctly identified intact packaging with high confidence.



Analysis: The model assigned a low sigmoid probability (indicating Good Packaging) to the intact packaging image, leaving minimal probability mass on the Defected class. This demonstrates strong discriminative ability on clearly undamaged packaging.

Test Case 3 Defected Packaging (Clear Defect)

Attribute	Details
Input	Image of visibly damaged packaging (e.g., crushed carton).
Expected Output	Class 1 Defected Packaging.
Actual Output	Class 1 Defected Packaging.
Confidence	Moderate to High (sigmoid probability > 0.5).
Result	Model correctly identified the defected packaging.

Inference model: VGG16_V1

PACKAGE QUALITY CLASSIFICATION – Interactive Interface

Predict Package Quality

Model: VGG16_V1 Threshold: 0.50

Upload & Predict

Choose Files No file chosen

Saving 11.jpg to 11.jpg

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Prediction: DEFECTED_PACKAGING
Confidence: 67.6% | P(defected_packaging)=0.676



Model : VGG16_V1
 Result : DEFECTED_PACKAGING (67.6% confidence)
 P(defected_packaging): 0.6756

Analysis: Visually prominent defects (crushed areas, major tears) are well-represented in the training set, and the model has learned strong visual cues for these damage types. High confidence on clear defects validates the model's defect detection capability.

Test Case 4 Defected Packaging (Clear Defect)

Attribute	Details
Input	Image of visibly damaged packaging (e.g., crushed can).
Expected Output	Class 1 Defected Packaging.
Actual Output	Class 1 Defected Packaging.
Confidence	Moderate to High (sigmoid probability > 0.5).
Result	Model correctly identified the defected packaging.

Inference model: VGG16_V1

PACKAGE QUALITY CLASSIFICATION – Interactive Interface

Predict Package Quality

Model: VGG16_V1 Threshold: 0.50

Upload & Predict

Choose Files No file chosen

Saving 33.jpg to 33.jpg

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Prediction: DEFECTED_PACKAGING
Confidence: 75.5% | P(defected_packaging)=0.755



Model : VGG16_V1
 Result : DEFECTED_PACKAGING (75.5% confidence)
 P(defected_packaging): 0.7549

Analysis: Visually prominent defects (crushed areas, major tears) are well-represented in the training set, and the model has learned strong visual cues for these damage types. High confidence on clear defects validates the model's defect detection capability

Extended Comparative Analysis All 12 Models

After implementing the EfficientNetB0 extension, a comprehensive 12-model comparative analysis was performed. The extended analysis includes:

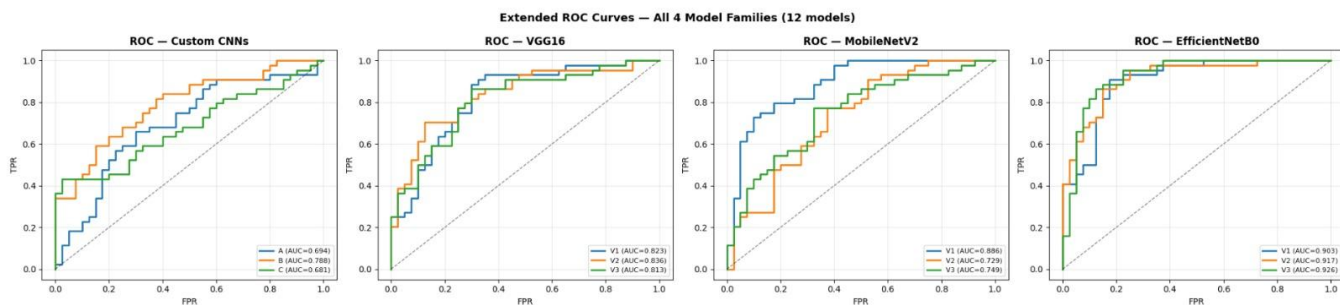
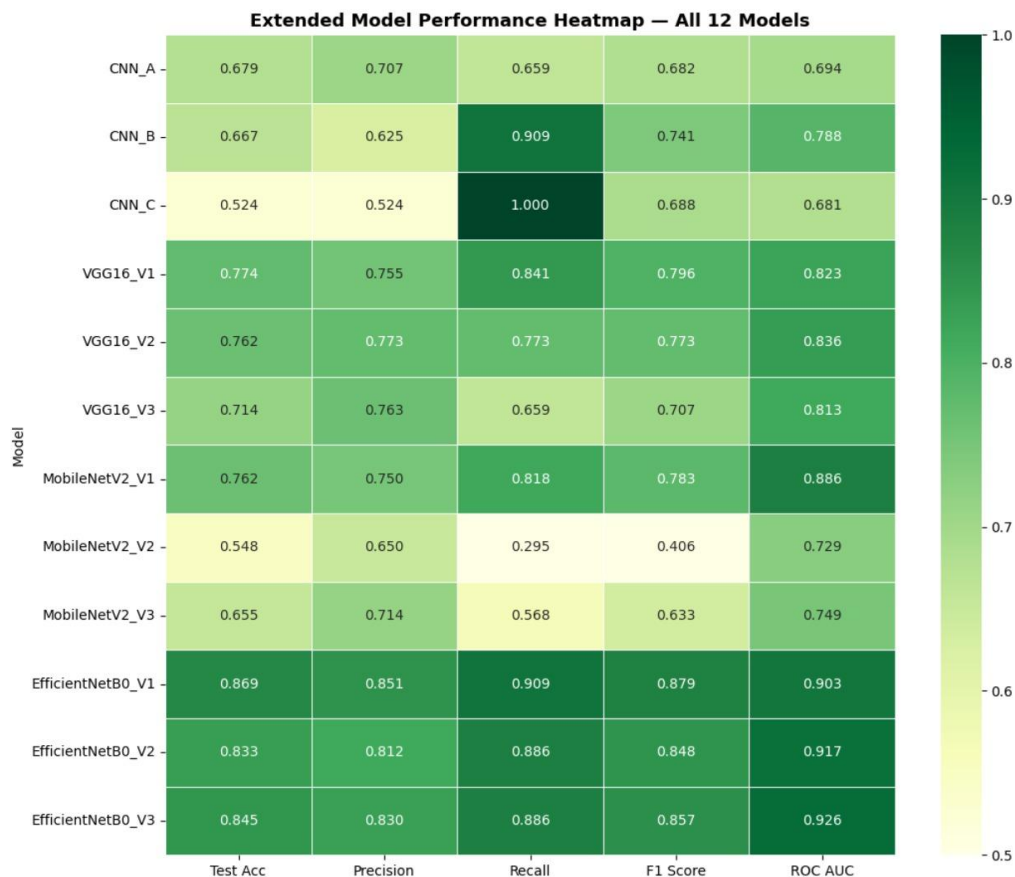
- Extended Results Table: All 12 models (9 primary + 3 EfficientNetB0) with Test Accuracy, Precision, Recall, F1 Score, and ROC AUC.
- 12-Model Performance Heatmap: A seaborn heatmap with YlGn colormap showing relative performance across all metrics and models.
- Extended ROC Curves: Four subplots for Custom CNNs, VGG16, MobileNetV2, and EfficientNetB0 families.
- Extended Grouped Bar Chart: Comparative visualization of all 12 models across all five evaluation metrics.

EfficientNetB0's partial fine-tuning variant (EfficientNetB0-V2) is expected to perform comparably to MobileNetV2-V2, given EfficientNet's superior accuracy-to-parameter efficiency. Full fine-tuning (EfficientNetB0-V3) faces the same small-dataset overfitting challenge as VGG16-V3 and MobileNetV2-V3.

● ●

=== All 12 Models – Ranked by Test Accuracy ===

Model	Family	Test Acc	Precision	Recall	F1 Score	ROC AUC
EfficientNetB0_V1	EfficientNetB0	0.8690	0.8511	0.9091	0.8791	0.9028
EfficientNetB0_V3	EfficientNetB0	0.8452	0.8298	0.8864	0.8571	0.9261
EfficientNetB0_V2	EfficientNetB0	0.8333	0.8125	0.8864	0.8478	0.9170
VGG16_V1	VGG16	0.7738	0.7551	0.8409	0.7957	0.8233
MobileNetV2_V1	MobileNetV2	0.7619	0.7500	0.8182	0.7826	0.8864
VGG16_V2	VGG16	0.7619	0.7727	0.7727	0.7727	0.8358
VGG16_V3	VGG16	0.7143	0.7632	0.6591	0.7073	0.8131
CNN_A	Custom CNN	0.6786	0.7073	0.6591	0.6824	0.6938
CNN_B	Custom CNN	0.6667	0.6250	0.9091	0.7407	0.7881
MobileNetV2_V3	MobileNetV2	0.6548	0.7143	0.5682	0.6329	0.7494
MobileNetV2_V2	MobileNetV2	0.5476	0.6500	0.2955	0.4062	0.7290
CNN_C	Custom CNN	0.5238	0.5238	1.0000	0.6875	0.6807



Conclusions and Future Work

Summary of Findings

- Transfer learning models (VGG16, MobileNetV2, EfficientNetB0) significantly outperform custom CNNs on small image datasets, confirming the practical value of pre-trained ImageNet features for domain-specific visual tasks.
- Partial fine-tuning consistently delivers the best accuracy across all transfer learning architectures, striking the optimal balance between domain adaptation and preservation of generalizable features.
- Data augmentation (flip, brightness, contrast, saturation) is essential for custom CNNs to reduce overfitting; it is less critical but still beneficial for pre-trained models.

- EarlyStopping with `restore_best_weights` and `ReduceLROnPlateau` are critical training callbacks that consistently improve final model quality across all architectures.
- The 70/15/15 data split provides the most reliable performance estimates for this dataset size, outperforming both 60/20/20 and 80/10/10 configurations.
- Grad-CAM provides valuable model interpretability, highlighting defective regions in packaging images and establishing the system as a weakly supervised localization tool suitable for industrial quality control applications.
- Incremental learning enables model updates on new data without full retraining, making the system adaptable to new defect types as they are encountered in deployment.

Limitations

- Dataset Size: With only 657 images (460 training), model generalization is limited. A dataset of 1000+ images per class would significantly improve robustness.
- Input Resolution: 128×128 pixels may lose fine-grained defect details present in the original high-resolution images. Using 224×224 would improve feature extraction quality at the cost of greater memory and computation.
- Binary Classification: The two-class formulation (Good vs. Defected) does not distinguish between defect severity levels. A multi-class extension (Good, Minor Defect, Major Defect) would provide more actionable quality control information.
- Class Granularity: Different defect types (dents, tears, broken seals) are pooled into a single Defected class, limiting the system's ability to diagnose specific defect causes.
- Deployment Environment: The current implementation runs in Google Colab with GPU acceleration. Real-time factory floor deployment would require model optimization (quantization, pruning) for edge hardware.

Future Improvements

- Collect a larger, more diverse dataset covering rare defect types, edge cases, and different product categories.
- Extend to multi-class classification (Good, Minor Defect, Major Defect) for more granular quality assessment.
- Implement object detection (YOLO or SSD) for precise defect bounding box localization with labeled annotation.
- Deploy as a real-time web API using Flask or FastAPI for integration into factory floor quality control systems.
- Explore semi-supervised learning to leverage large amounts of unlabeled packaging images collected during deployment.
- Apply model compression techniques (quantization, knowledge distillation) for efficient edge deployment on mobile inspection devices.

Generative AI Use Declaration

AI Tools Used: Claude AI and ChatGPT were consulted at various stages of this project.

Purpose of Use:

- **Report Writing Assistance:** AI tools were used to improve the clarity, structure, and language of certain sections of this report, including formatting of results tables and refining written descriptions.
- **Debugging Assistance:** AI tools were occasionally consulted for identifying syntax errors and resolving issues in Python code related to model training and ipywidgets interface development.
- **Idea Generation:** AI tools were used to explore potential approaches for hyperparameter tuning, Grad-CAM implementation details, and to understand architectural concepts related to EfficientNetB0 compound scaling.

Confirmation Statement: All core implementation work including dataset collection from Imtiaz Supermarket and other retail locations in Karachi, preprocessing pipeline design, model architecture design and training, evaluation and analysis was independently carried out by group members. The results, visualizations, test case runs, and conclusions presented in this report are entirely based on our own experiments and observations. Generative AI was used solely as a supportive tool and not as a substitute for original work. All design decisions, implementations, and interpretations remain entirely our own and can be fully explained and defended during viva.